

コマ 10: Type + ポインタ演算 + 配列

今日のゴール

型サイズを管理する `Type` を導入し、配列アクセスとポインタ演算を実装する。

第 09 回では、ポインタも整数もすべて 8 バイト値として扱った。しかし、配列やポインタ演算を扱うには「指している先の型サイズ」が必要になる。

```
int *p;  
p + 2;    // 2 バイト進むのではなく、2 * sizeof(int) バイト進む
```

この回では、次の 2 つを実装する。

機能	例
固定長配列	<code>int a[5]; a[i]</code>
ポインタ演算	<code>p + 2, p - 1</code>

1 この回で扱う範囲

対象にする型は、`int`、`char`、ポインタ、固定長配列である。

型	サイズ
<code>int</code>	4 バイト
<code>char</code>	1 バイト
<code>T *</code>	8 バイト
<code>T[N]</code>	<code>sizeof(T) * N</code> バイト

この回では、配列はローカル変数としてのみ扱う。文字列リテラル、`malloc`、構造体、グローバル変数は後の回で扱う。

2 AST を確認する: 配列アクセス

まず、配列を使うプログラムがどのような AST になるか確認する。

```
python3 scaffold/parse_viewer.py sessions/10_types_arrays/tests/array_sum.c
```

このプログラムの内容は次の通り。

```
int main() {
    int a[5];
    int i;
    int sum;
    a[0] = 1;
    a[1] = 2;
    a[2] = 3;
    a[3] = 4;
    a[4] = 5;
    sum = 0;
    for (i = 0; i < 5; i = i + 1) {
        sum = sum + a[i];
    }
    return sum;
}
```

実行すると、次のような S 式が表示される。

```
(program
 (funcdef "main" :type int (params)
  (block
   (decl "a" :type (array int 5))
   (decl "i" :type int)
   (decl "sum" :type int)
   (exprstmt
    (assign
     (index (var "a") (num 0))
     (num 1)))
   (exprstmt
    (assign
     (index (var "a") (num 1))
     (num 2)))
   (exprstmt
```

```

      (assign
        (index (var "a") (num 2))
        (num 3)))
(exprstmt
  (assign
    (index (var "a") (num 3))
    (num 4)))
(exprstmt
  (assign
    (index (var "a") (num 4))
    (num 5)))
(exprstmt
  (assign (var "sum") (num 0)))
(for
  (init
    (assign (var "i") (num 0)))
  (cond
    (lt (var "i") (num 5)))
  (step
    (assign (var "i")
      (add (var "i") (num 1)))))
  (body
    (block
      (exprstmt
        (assign (var "sum")
          (add (var "sum")
            (index (var "a") (var "i"))))))))
  (return (var "sum"))))

```

`a[i]` はAST上では `(index (var "a") (var "i"))` になる。このノードは、`*(a + i)` とほぼ同じ意味で扱う。

3 ASTを確認する: ポインタ演算

次に、配列の先頭アドレスをポインタに代入し、ポインタ演算で読む例を確認する。

```
python3 scaffold/parse_viewer.py sessions/10_types_arrays/tests/ptr_arith.c
```

このプログラムの内容は次の通り。

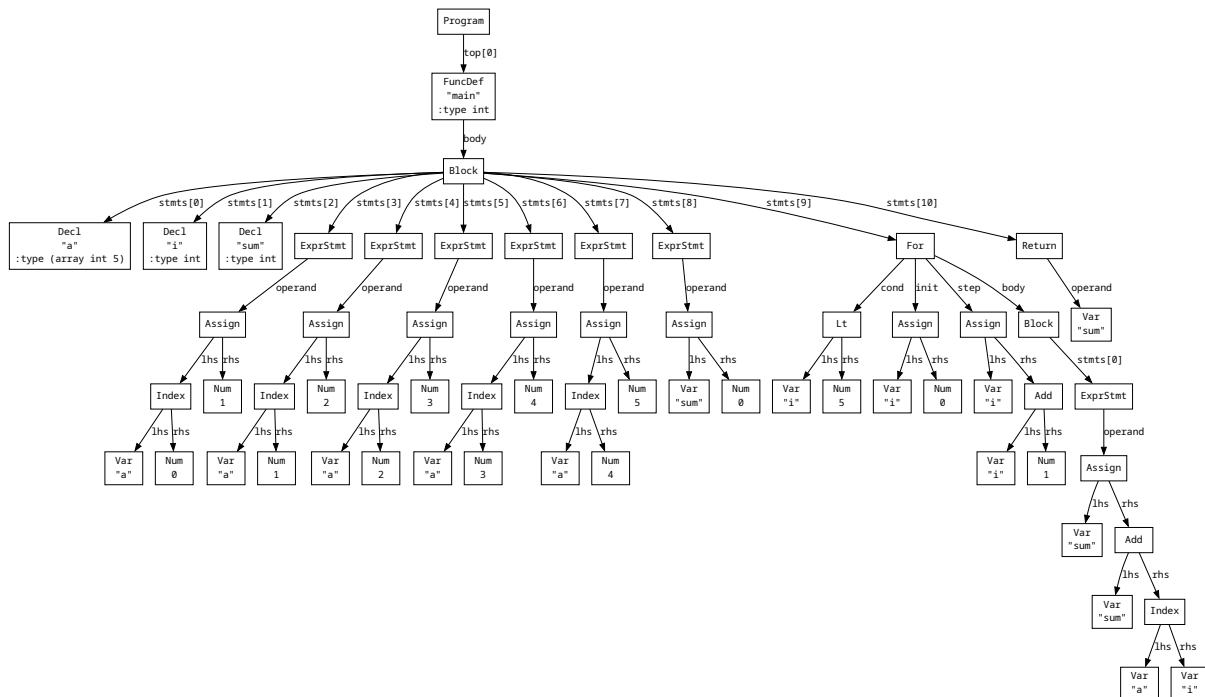


図 1 array_sum.c の AST

```

int main() {
    int a[4];
    int *p;
    a[0] = 10;
    a[1] = 20;
    a[2] = 30;
    a[3] = 40;
    p = a;
    return *(p + 2) + *(p + 3);
}

```

実行すると、次のような S 式が表示される。

```

(program
  (funcdef "main" :type int (params)
    (block
      (decl "a" :type (array int 4))
      (decl "p" :type (ptr int))
      (exprstmt
        (assign
          (index (var "a") (num 0))
          (num 10)))

```

```

(exprstmt
  (assign
    (index (var "a") (num 1))
    (num 20)))
(exprstmt
  (assign
    (index (var "a") (num 2))
    (num 30)))
(exprstmt
  (assign
    (index (var "a") (num 3))
    (num 40)))
(exprstmt
  (assign (var "p") (var "a")))
(return
  (add
    (deref
      (add (var "p") (num 2)))
    (deref
      (add (var "p") (num 3))))))

```

`p + 2` は、単にアドレスに 2 を足すのではない。`p` は `int *` なので、`2 * sizeof(int)`、つまり 8 バイト進む。

4 型の扱い

新しいスキファールドでは、`Type` クラスを使わず、`ty_str` 文字列と `self._struct_defs` を組み合わせて型を扱う。

型サイズが必要なときは、次のヘルパーメソッドを使う。

メソッド	役割
<code>self.size_of_ty_str(ty)</code>	<code>ty_str</code> から型のバイトサイズを返す
<code>self.elem_ty_str(ty)</code>	ポインタ・配列の要素型の <code>ty_str</code> を返す

メソッド	役割
<code>self.base_ty_str(ty) /</code> <code>self.deref_ty_str(ty)</code>	ポインタの参照先型／逆参照

`ty_str` の例:

C コード	<code>ty_str</code>
<code>int a;</code>	<code>int</code>
<code>char c;</code>	<code>char</code>
<code>int *p;</code>	<code>int*</code>
<code>int a[5];</code>	<code>int[5]</code>

`self.alloc_local()` では、`ty_str` から `self.size_of_ty_str()` を使って必要なスタック領域を計算する。

5 配列変数の扱い

通常の変数を `rvalue` として使うときは、変数のアドレスから値を `ld / lw` で読む。しかし配列変数を `rvalue` として使うときは、配列全体を読み込むのではなく、先頭要素のアドレスを返す。

```
int a[4];
int *p;
p = a;    // a は配列先頭アドレスとして使われる
```

つまり `codegen(ND_VAR)` は、型が配列かどうかで処理を分ける。

```
if node.kind == ND_VAR:
    self.codegen_lval(node)
    if self._type_of_expr(node) の ty_str が '[...]' を含まない:
        self._emit_load(ty)
    return
```

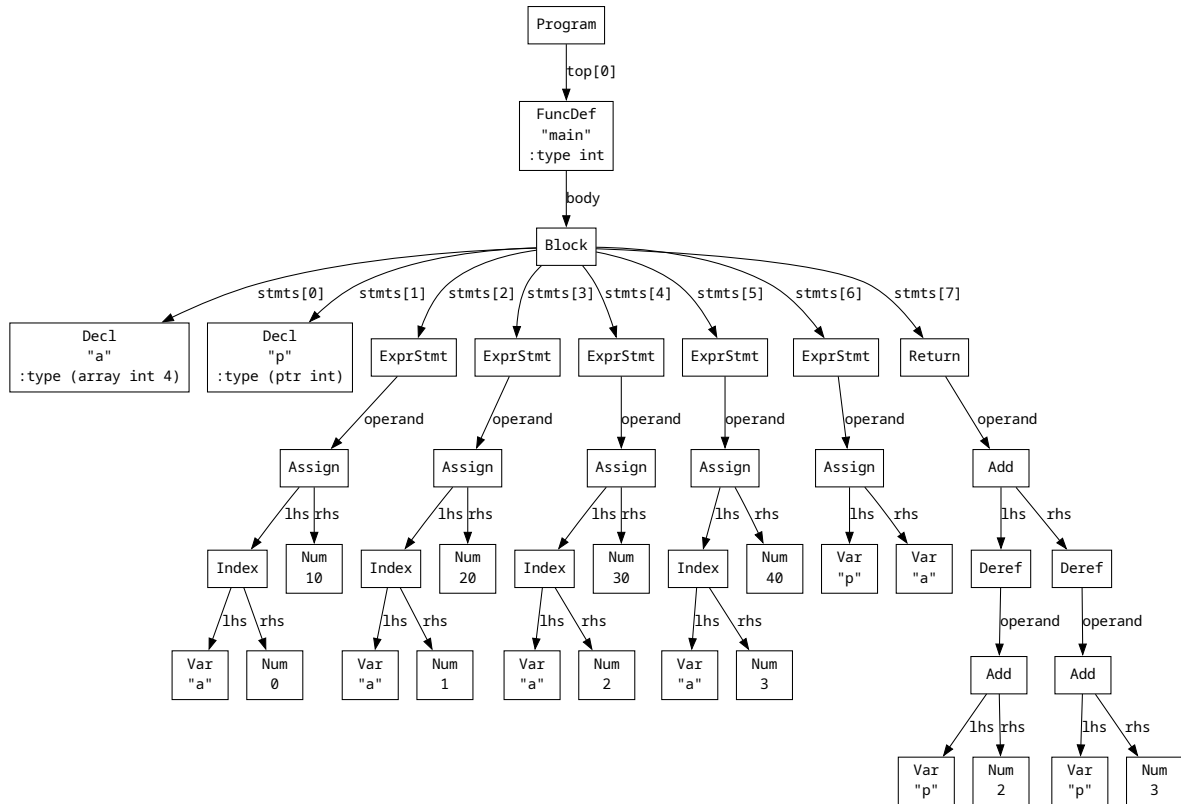


図 2 ptr_arith.c の AST

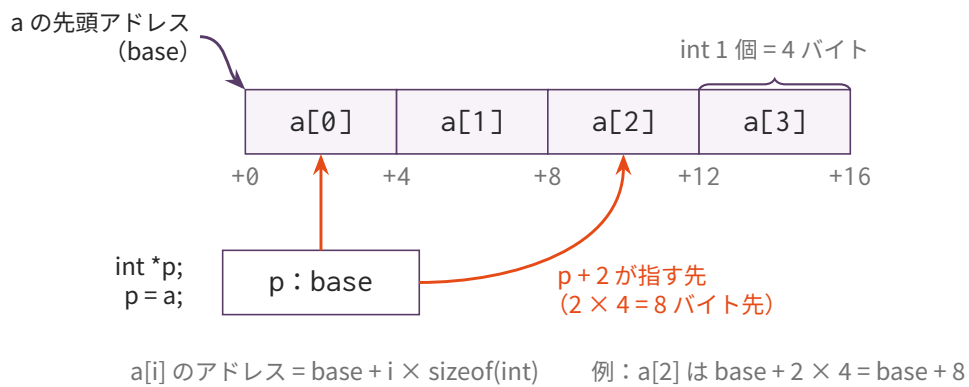


図 3 int a[4] のメモリ配置と a[i] / p + 2 のアドレス計算

6 a[i] のコード生成

a[i] は、配列またはポインタの先頭アドレスに $i \times \text{要素サイズ}$ を足した場所を表す。

$$\text{address}(a[i]) = \text{address}(a) + i * \text{sizeof}(\text{element})$$

要素 1 つは `sizeof(int) = 4` バイトなので、`p + 2` はアドレスを 8 バイト進める。この倍率を決めるために、要素型のサイズが必要になる。

lvalue としての `a[i]` は、次の流れでアドレスを作る。

```
self.codegen(node.lhs)    # a の先頭アドレス、または p の値
push a0
self.codegen(node.rhs)    # 添字 i
scale by element size
pop a1
add a0, a1, a0             # a0 = base + i * size
```

rvalue としての `a[i]` は、このアドレスから値を読む。

7 ポインタ演算

`p + n` では、`n` に要素サイズを掛けてからアドレスに加える。

式	実際に足す値
<code>int *p; p + 2</code>	<code>2 * 4</code>
<code>char *p; p + 2</code>	<code>2 * 1</code>
<code>int **p; p + 2</code>	<code>2 * 8</code>

`self.elem_ty_str()` がないと、この倍率を決められない。

8 _emit_load / _emit_store

型サイズに応じて、読み書きする命令を変える。

```
def _emit_load(self, ty):
    size = self.size_of_ty_str(ty)
    if size == 1:
        self.emit(" lb a0, 0(a0)")
    elif size == 4:
```



```

        self.emit(" lw a0, 0(a0)")
    else:
        self.emit(" ld a0, 0(a0)")

    def _emit_store(self, ty):
        size = self.size_of_ty_str(ty)
        if size == 1:
            self.emit(" sb a0, 0(a1)")
        elif size == 4:
            self.emit(" sw a0, 0(a1)")
        else:
            self.emit(" sd a0, 0(a1)")

```

第 09 回では常に `ld` / `sd` でよかったが、この回からは型サイズを見て命令を選ぶ。

9 編集するファイル

- `mycc.py`

`importlib` でコマ 09 の `Codegen09` を継承した `Codegen10` に、以下の機能を追加する (スケルトンにあらかじめ書かれている)。

ハンドラメソッド	変更内容
<code>_emit_load(ty) / _emit_store(ty)</code>	型サイズに応じて <code>lb/lw/ld</code> と <code>sb/sw/sd</code> を選ぶ
<code>self.size_of_ty_str(ty)</code>	<code>ty_str</code> から型のバイトサイズを計算する
<code>self.elem_ty_str(ty)</code>	ポインタ・配列の要素型 <code>ty_str</code> を返す
<code>codegen(node)</code> の <code>match</code> 節に <code>'Index'</code>	<code>a[i]</code> のアドレス計算とロード
<code>codegen(node)</code> の <code>match</code> 節 <code>'Var'</code>	配列型の場合 <code>lval</code> アドレスだけ返しロードしない
<code>codegen(node)</code> の <code>match</code> 節 <code>'Add' / 'Sub'</code>	ポインタ演算：添字に要素サイズを掛ける

10 実装手順

1. スケルトンの `Codegen10` が `Codegen09` を `importlib` で継承していることを確認する
2. `_emit_load` / `_emit_store` を型サイズ対応にする (`size_of_ty_str` を使う)
3. `self.alloc_local()` で `ty_str` から `size_of_ty_str` を呼んでスタック領域を決める
4. `codegen_lval(node)` に 'Index' handlerを追加する (`a0 = base + i * elem_size`)
5. `codegen(node)` の 'Index' handlerを追加する (`lval` → `_emit_load`)
6. `codegen(node)` の 'Var' handlerで配列型はロードをスキップする
7. `codegen(node)` の 'Add' / 'Sub' handlerでポインタ演算を追加する

11 テスト

```
python3 scaffold/test_runner.py sessions/10_types_arrays
```

この回の主要テストは次の通り。

テスト	内容	期待値
<code>array_sum.c</code>	<code>a[i]</code> で配列合計	15
<code>ptr_arith.c</code>	<code>*(p + 2)</code> と <code>*(p + 3)</code>	70